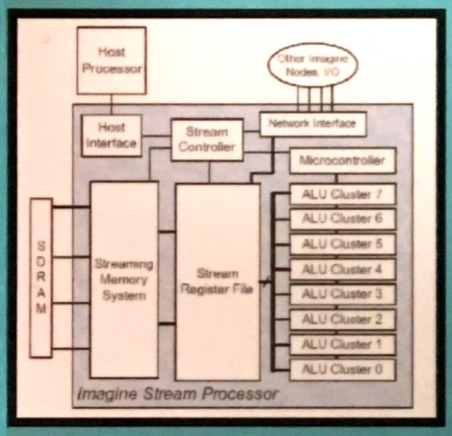
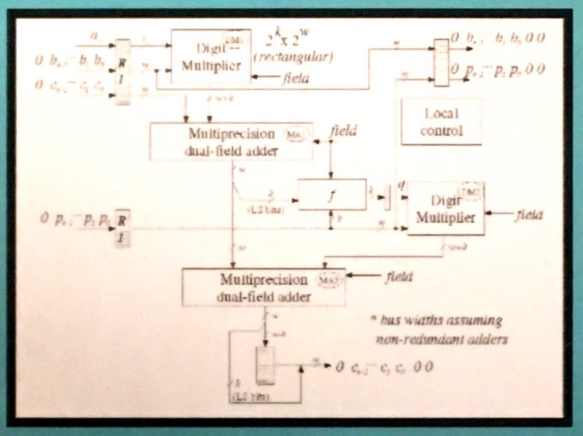
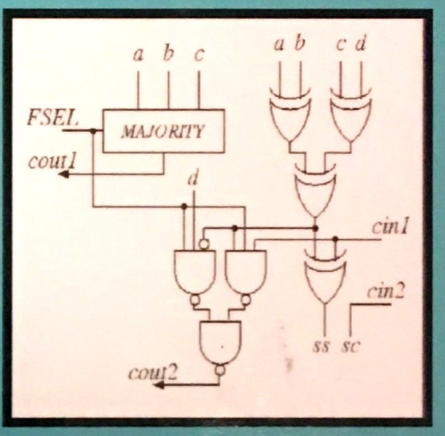
Nadia Nedjah
Luiza de Macedo Mourelle (Editors)

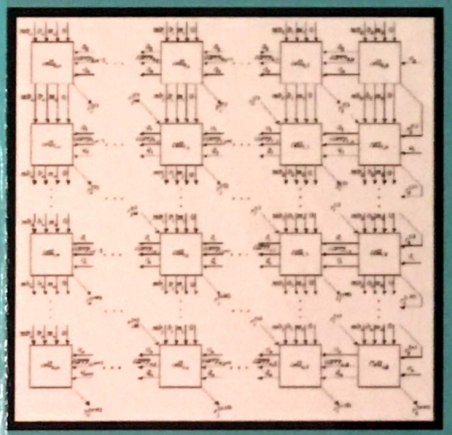
EMBEDDED CRYPTOGRAPHIC HARDWARE

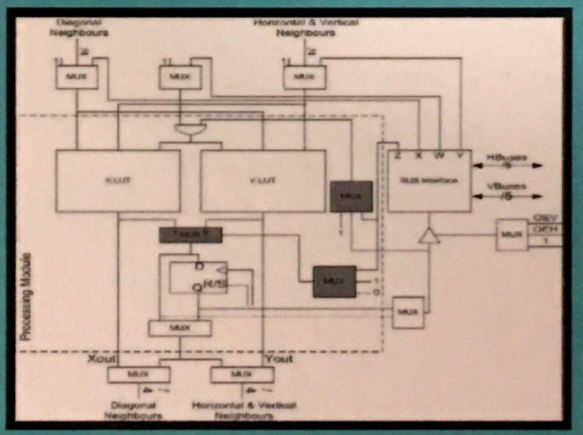
Methodologies and Architectures

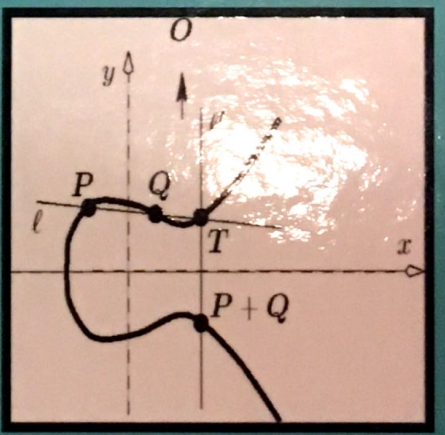


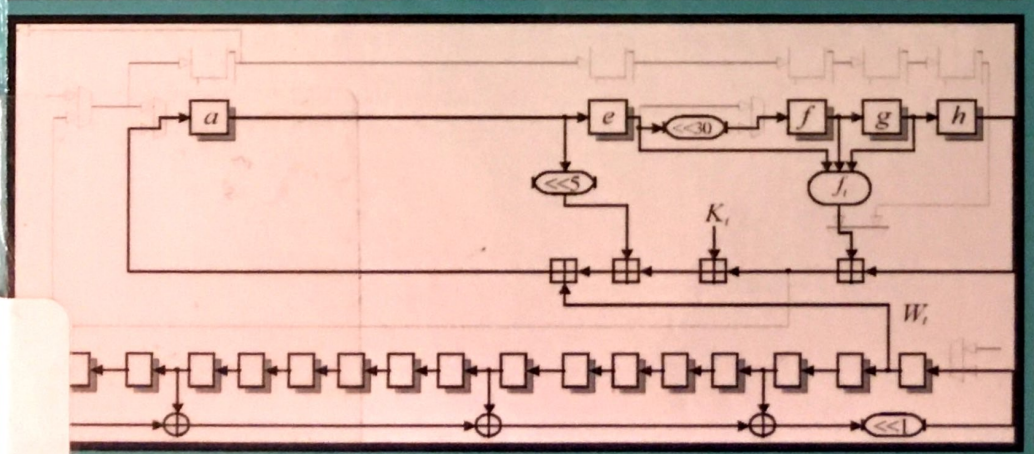


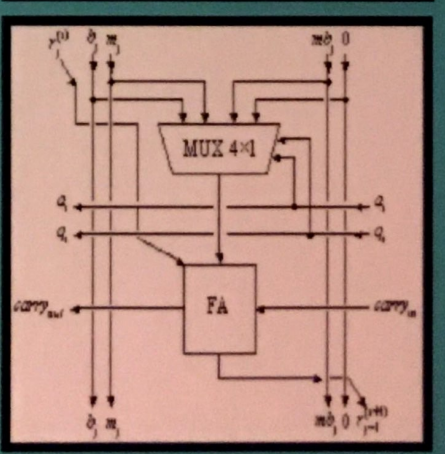












NOVA

Production Coordinator: Donna Dennis
Coordinating Editor: Tatiana Shohov
Senior Production Editors: Susan Boriotti and Donna Dennis
Production Editors: Marius Andronic and Rusudan Kazmadze
Office Manager: Annette Hellinger
Graphics: Levani Chiaidze
Editorial Production: Maya Columbus, Aldysha Klestov, Vladimir Klestov, Matthew Kozlowski and Lorna Loperfido
Circulation: Ave Maria Gonzalez, Vera Popovic, Luis Aviles, Jonathan Boriotti, Alexandra Columbus, Raymond Davis, Cathy DeGregory, Melissa Diaz, Magdalena Nuñez, Marlene Nuñez, Jeannie Pappas and Frankie Punger
Communications and Acquisitions: Serge P. Shohov

✱

Library of Congress Cataloging-in-Publication Data
Embedded cryptographic hardware : methodologies & architectures / Nadia Nedjah and Luiza de Macedo Mourelle (editors).
p. cm.
Includes index.
ISBN 1-59454-012-8 (hardcover)
1. Telecommunications--Security measures. 2. Data encryption (Computer science). 3. Computer security. 4. Embedded computer systems. I. Nedjah, Nadia. II. Macedo, Mourelle, Luiza de.
TK5102.94.E49 2004
005.8'2--dc22 2004008204

400 Oser Ave, Suite 1600
Hauppauge, New York 11788-3619
Tele. 631-231-7269 Fax 631-231-8175
e-mail: Novascience@earthlink.net
Web Site: http://www.novapublishers.com

Printed in the United States of America

CONTENTS

Preface		**vii**
Chapter I	Unified Hardware Architecture for the Secure Hash Standard *Akashi Staoh*	**1**
Chapter II	A Hardware Accelerator for Elliptic Curve Cryptography over GF(2^m) *Joseph Riley and Michael J. Schulte*	**17**
Chapter III	Two New Algorithms for Modular Multiplication *Viktor Bunimov and Manfred Schimmler*	**39**
Chapter IV	AES Packet Encryption on a SIMD Stream Processor *Yu-Kuen Lai and Gregory T. Byrd*	**57**
Chapter V	High-Speed Cryptography *Bertil Schmidt, Manfred Schimmler and Heiko Schröder*	**77**
Chapter VI	A Design Framework for Scalable and Unified Multipliers in $GF(p)$ and $GF(2^m)$ *Alexandre F. Tenca, Erkay Savaş and Çetin K. Koç*	**97**
Chapter VII	Three Hardware Implementations for the Binary Modular Exponentiation *Nadia Nedjah and Luiza de Macedo Mourelle*	**119**
Chapter VIII	An on Chip CAST-128 Based Block Cipher with Dynamically Reconfigurable Sboxes Generated in Parallel *Panayiotis E. Nastou and Yiannis C. Stamatiou*	**135**
Chapter IX	An FPGA-Based Design Flow of Five Cryptographic Algorithms *Juan M. Díez, Slobodan Bojanić, Carlos Carreras and Octavio Nieto-Taladriz*	**153**
Chapter X	A Carry-Free Montgomery Inversion Algorithm *Erkay Savaş*	**171**

In: Embedded Crytographic Hardware
Editors: N. Nedjah and L. Mourelle, pp. 39-55

ISBN 1-59454-012-8

Chapter III

TWO NEW ALGORITHMS FOR MODULAR MULTIPLICATION

Viktor Bunimov and Manfred Schimmler
Institute of Computer and Communication
Network Engineering
Technical University of Braunschweig
Hans-Sommer-Str. 66, 38106 Braunschweig,
GERMANY
v.bunimov@tu-bs.de

Two new algorithms for modular multiplication and their corresponding architectures are presented. These algorithms are optimizations of two well-known algorithms for modular multiplication: Montgomery multiplication and interleaved modular multiplication. They are optimized with respect to area and time complexity. In both algorithms the product of two n-bit-integers X and Y modulo M is computed by n iterations of a simple loop. Each loop consists of one single carry save addition, a comparison of constant complexity, and a table lookup. In each iteration of the loop, one of the following alternatives must hold: The current digit x_i of X can be 0 or 1 and the current intermediate sum can be odd or even. This leads to four possible combinations. Fore each of those there is a fixed value to be added to intermediate sum. The idea is now to compute these for values prior to the execution of the loop and store then in a lookup table. By these constructions the arithmetical complexity of the modular multiplication is reduced to $2n$ additions for Montgomery and n additions for interleaved modular multiplication without carry propagation, which leads to a speedup of at least two in comparison with all methods previously known. The evaluation of the new algorithms and the comparison with other methods is based on a simple complexity model for area and time, which is also presented in the chapter.

III.1. Introduction

Modular multiplication is a central operation in many application areas including public key cryptography. Given a word length of n bits, an n-bit integer M, called the modulus, and two n-bit operands X and Y, the problem is the computation of $X \times Y \bmod M$. Most devices in which this problem is essential have strict limitations in chip area and power consumption, and the typical problem size n is rather large (e.g. n=1024). Therefore, there is a need for algorithms that are on the one hand fast and on the other hand area and power efficient if implemented in hardware. An appropriate complexity measure for these algorithms is the product of area and time, AT. There is a variety of solutions with an optimal AT complexity of $O(n^2)$. But not only the asymptotic complexity is interesting for the application but also the constant factor. This is the reason why in almost all applications of modular multiplication the algorithms of Montgomery [15] and the interleaved modular multiplication [2], [21] are chosen.

Many researchers have found improvements to Montgomery [1], [3], [4], [7], [9], [20], [22] and to interleaved [2], [19], [21] modular multiplication. Some of them [7], [10], [12], [18] are significant better than other algorithms [5], [11], [20], [23], [24] with respect to area and time. The best optimizations of Montgomery and interleaved modular multiplication require at least n iterations of an inner loop with not less than 2 additions in each iteration. The aim of this chapter is the optimization of Montgomery and interleaved modular multiplication using lookup tables, redundant addition, approximative comparison and other techniques, in order to reduce the number of additions to 1 addition with constant time per iteration, without increasing of area complexity. The new algorithms are designed for practical implementations and can be easily realized as full custom circuits or FPGAs.

This chapter is organized as follows. The carry save addition and redundant number representation is presented in Section III.2 together with our model for evaluation of time and area. In Section III.3, Montgomery modular multiplication and interleaved modular multiplication algorithms are explained together with the complexity analysis. The optimization of Montgomery modular multiplication is given in Section III.4. Section III.5 describes an optimized version of the interleaved modular multiplication algorithm and its complexity analysis. Section III.6 shows the comparison of these algorithms with respect to area and time complexity. Finally, Section III.7 gives some outlook to future work and concludes the chapter.

III.2. Carry Save Adders, Redundant Representation, and Complexity Model

The central operation of most algorithms for modular multiplication including ours is addition. There are many different methods for addition: ripple carry addition, carry select addition, carry lookahead addition and others [13]. The disadvantage of all these methods is the carry propagation, which makes the latency of the addition dependent on the length of the operands. This is not a big problem for operands of size 32 or 64 but the typical operand size in cryptographic applications is between 1024 and 8192. The resulting delay has a significant influence on the time complexity.

Carry save addition [12] is a method for an addition without carry propagation. It is simply a parallel ensemble of n full-adders without any interconnection. Its function is to add three n-bit integers X, Y, and Z to produce two integers C and S as results such that

$$C + S = X + Y + Z \tag{1}$$

The i^{th} bit of the sum si and the $(i+1)^{st}$ bit of carry c_{i+1} is calculated using the Boolean equations

$$s_i = x_i \oplus y_i \oplus z_i$$

$$c_{i+1} = x_i y_i \vee x_i z_i \vee y_i z_i \tag{2}$$

$$c_0 = 0,$$

When carry save adders are used in an algorithm one uses a notation of the form

$$(S, C) = X + Y + Z \tag{3}$$

to indicate that two results are produced by the addition.

The results are now represented in two binary words, an n-bit word S and an $(n+1)$-bit word C. Of course, this representation is redundant in the sense that we can represent one value in several different ways. This redundant representation has the advantage that the arithmetic operations are fast, because there is no carry propagation. On the other hand, there is a disadvantage necessarily coming with redundant number representations: Comparison between numbers and even comparison to 0 is expensive with respect to area and time. But comparison is an essential part of many modular multiplication algorithms [5], [2], [15], [21]. In Sections III.4 and III.5 we will show how we can overcome this problem for Montgomery multiplication and interleaved modular multiplication.

For comparison of different algorithms we need a complexity model that allows a realistic evaluation of time and area requirements of the considered methods. We take the delay of a full adder (1 time unit) as a reference for time requirements and quantify the delay of an access to a (small) lookup table with the same time delay of 1 time unit. Our area estimation is based on empirical studies in full-custom and semi-custom layouts for adders and storage elements: The area for 1 bit in a lookup table corresponds to 1 area unit. A register cell requires 4 area units per bit and a full adder requires 8 area units. These values provide a powerful and realistic model for evaluation of area and time for most algorithms for modular multiplication.

III.3. Montgomery and Interleaved Modular Multiplication Algorithms

The idea of Montgomery is to keep the lengths of the intermediate results smaller than $n+1$ bits. This is achieved by interleaving the computations and additions of new partial

products with divisions by 2, each of them reducing the bit-length of the intermediate result by one.

Algorithm III.1a. Montgomery multiplication
Inputs: X,Y,M with $0 \le X,Y < M$
Output: $P = (X \times Y \times (2^n)^{-1}) \bmod M$
n: number of bits in X,
x_i : i^{th} bit of X
p_0: LSB of P
(1) $P := 0$;
(2) for (i=0; i<n; i++) {
(3) $P := P + x_i * Y$;
(4) $P := P + p_0 * M$;
(5) $P := P$ div 2 }
(6) if ($P \ge M$) then $P := P - M$;

The basic idea of Montgomery [3], [9], [20], [22] is the following: adding a multiple of M to the intermediate result does not change the value of the final result, because the result is computed modulo M. M is an odd number. Therefore, after each addition in the inner loop the least significant bit (LSB) of the intermediate result is inspected. If it is 1, i.e. the intermediate result is odd, we add M to make it even. This even number can be divided by 2 without remainder. This division by 2 reduces the intermediate result to n+1 bits again. After n steps these divisions add up to one division by 2^n. Montgomery Multiplication requires two passes through the same multiplication process, therefore doubling the computation time. The first pass computes $P = (X*Y*(2^n)^{-1}) \bmod M$ and the second pass computes $(P*2^{2n}*(2^n)^{-1}) \bmod M = (X*Y) \bmod M$ which is the desired result. On the other hand, it is very easy to implement since it operates least significant bit first and does not require any comparisons. Therefore, it can be implemented using carry save adders and a redundant representation of the intermediate results. A modification of Montgomery's Algorithm with carry save adders is given in Algorithm III.1b:

Algorithm III.1b. Fast Montgomery multiplication
Inputs: X,Y,M with $0 \le X,Y < M$
Output: $P = (X \times Y \times (2^n)^{-1}) \bmod M$
n: number of bits in X,
x_i : i^{th} bit of X
s_0: LSB of S
(1) $S := 0$; $C := 0$;
(2) for (i=0; i<n; i++) {
(3) $S,C := S + C + x_i * Y$;
(4) $S,C := S + C + s_0 * M$;
(5) $S := S$ div 2; $C := C$ div 2; }
(6) $P := S + C$
(7) if ($P \ge M$) then $P := P - M$;

In this algorithm the delay of one pass through the loop is reduced from O(log n) to O(1). We take the notion of an assignment of the sum of three operands to two values S, C as in order to indicate the use of a carry save adder.

Of course, the additions in step 6 and 7 are conventional additions. But since they are performed only once while the additions in the loop are performed n times this is subdominant with respect to the time complexity.

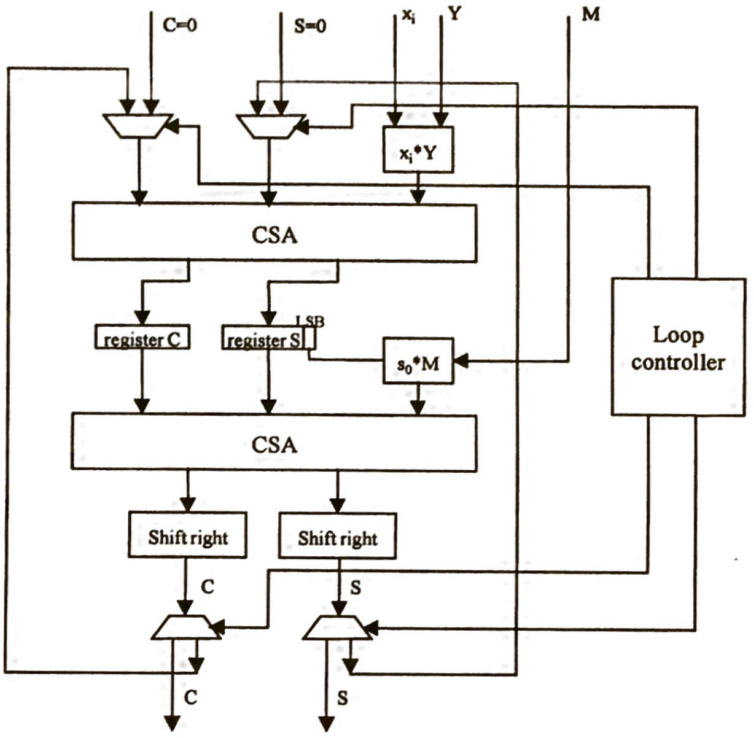


Figure III.1. Implementation of the loop of Algorithm III.1b

Figure III.1 shows an architecture for the hardware implementation of Algorithm III.1b. The dominant parts both for area and time are the two carry save adders and 5 registers required to compute the values of S and C in the loop of Algorithm III.2. Each carry save adder consists of n+1 full adders. The area complexity of the algorithm 2 is $2*8*n + 5*4*n = 36n$. The time complexity is $2*2*n=4n$ (2 loops with n iterations each with 2 carry save additions). The area time complexity is

$$AT = 36n*4n = 144n^2. \quad (4)$$

In Section III.4 we will show how this can be improved to only one carry save adder within the loop.

Redundant number representation is used in other optimisations of Modular multiplication of Montgomery [9] too. The authors use numbers over the alphabet of three digits {0, 1, 2}. ($X*Y$) mod M is computed using $5n$ half adders and $3n$ Or-gates. If one full adder requires 8 area units and one n-bits carry save adder requires $8n$ area units, then we can estimate the area complexity of one new adder 20n. The algorithm requires 6 additional n-bit registers. The area complexity of 6 registers is $6*4n=24n$. The complete area complexity is

$20n+24n=44n$. The time complexity of one addition can be estimated as twice the time complexity of one carry save adder. This means, that the time complexity of one addition is equal to 2. One loop of this algorithm requires roughly n iterations. Thus, the time complexity of this algorithm is equal to $2n$. Consequently, the time complexity of the modular multiplication is $4n$. The area complexity is $20n$. Thus, the area time complexity is $4n*44n=176n^2$.

The other technique to speed up Montgomery modular multiplication is the use of high radix. This technique is used for example in [25]. The authors use the radix 4, i.e. blocks of two bits are computed together. In this chapter the Booth algorithm is used. The area complexity of this algorithm can be estimated as $3n$ full adders and more than 10 n-bit registers. Using our area complexity model, the area complexity of this algorithm is greater than $3n*8 + 10n*4 = 64n$ area units. The time complexity of one iteration of the loop is equal to 2. The loop consists of $n/2$ iterations, and the algorithm requires 2 loops for one modular multiplication. Thus, the time complexity of this algorithm is equal $2*n/2*2=2n$ time units. Consequently, the AT-complexity of the algorithm is greater than $64n*2n=128n^2$. The implementations with higher radices speed up the algorithms but they increase the area complexity.

The second algorithm for modular multiplication is the interleaved modular multiplication. The idea is to interleave multiplication and reduction such that the intermediate results are kept as short as possible. This is reached by doing one step of the multiplication at a time (more precisely: one bit x_i of the operand X is multiplied with the whole operand Y) which is added to the old intermediate result. The sum is then reduced with respect to M by subtracting M until the remainder is smaller than M was producing the next intermediate result P. The loop invariant for the inner loop is $P<M$, which holds before (3) and after (7). After step (5) the value of P is smaller than $3M$. Therefore two subtractions of M in steps (6) and (7) are sufficient to keep the loop invariant true. After this reduction of P the next bit of X is processed.

Algorithm III.2. Standard interleaved modulo multiplication
Input: X, Y, M with $0 \leq X,Y \leq M$
Output: $P = X*Y \bmod M$
n: number of bits in X
x_i: i^{th} bit of X
(1) P:=0;
(2) for (i=n-1; i≥0; i--){
(3) P=2*P;
(4) I=x_i*Y;
(5) P:=P+I;
(6) If ($P \geq M$) Then P:=P-M;
(7) If ($P \geq M$) Then P:=P-M;}

Figure III.2 shows the architecture of the algorithm for interleaved multiplication:

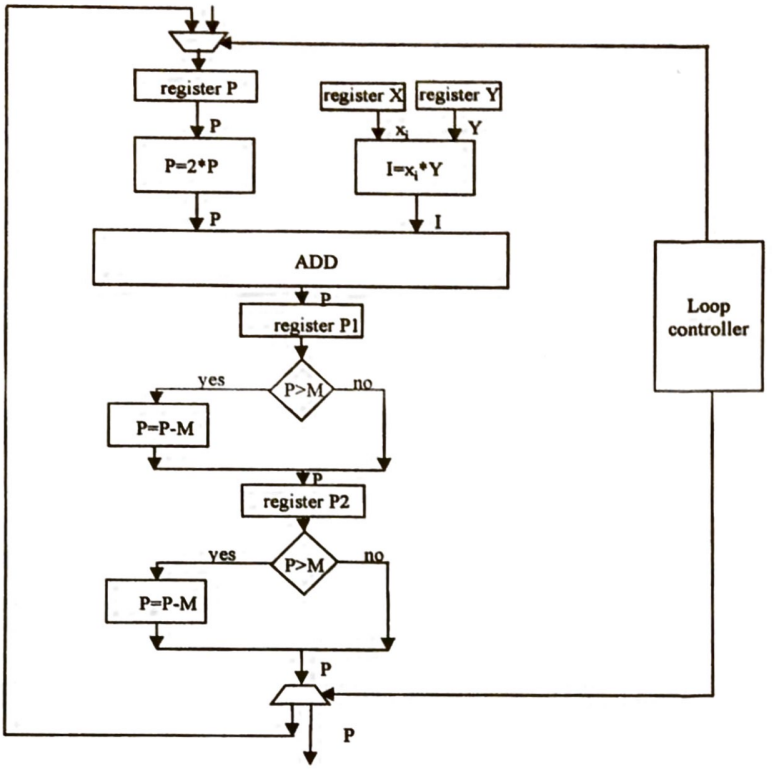


Figure III.2. Interleaved modular multiplication

The advantages of Algorithm III.2 in contrast to the separated multiplication and division are the following:

- Only one loop is required for the whole operation.
- The intermediate results are never any longer than n+2 bits (thus reducing the area for registers and full adders).

But there are some disadvantages as well:

- The algorithm requires three additions with carry propagation in steps (5), (6) and (7) (computing $2P$ is only a shift operation).
- In order to perform the comparisons in steps (4) and (5), the preceding additions have to be completed. This is important for the latency because the operands are large and, therefore, the carry propagation has a significant influence on the latency.
- The comparison in step (6) and (7) also requires the inspection of the full bits lengths of the operands in the worst case. In contrast to addition, the comparison is performed MSB-first. Therefore, these two operations cannot be pipelined without delay.

Several researcher have tried to overcome these problems [12], [17], [19]. But the only solution with a constant delay in the loop is the one of [12], which has an AT-complexity of $156n^2$.

In the following we describe a different approach, reducing the AT-complexity for modular multiplication to $28n^2$.

Its time complexity is determined by the number n of iterations of the loop and the time for the additions in steps (5), (6), and (7). Assuming the additions are performed by carry lookahead adders with the optimal latency of O(log n) the time complexity is O(n log n). We simplify the evaluation of the area by only counting the required full adders for executing steps (5), (6), and (7). Since a carry lookahead adder has an area complexity roughly comparable to a carry ripple adder we can state the area as O(n) [8]. For the product of area and time we get an asymptotic complexity

$$AT = O(n^2 \log n) \tag{5}$$

for the interleaved modular multiplication with Algorithm III.2.

It becomes more complicated if we want to estimate the constant factor. The time requirements can be determined as the number of subsequent complete additions where we take the delay of a full adder as one time unit. In this case the latency of an n-bit carry lookahead adder can be counted as 2*(log n). Since each addition in steps (5), (6), and (7) must be completed before the subsequent steps can be executed we result in 6log n for one iteration and $T = 6n \log n$ for Algorithm III.2.

Algorithm III.2 requires at least one n-bit carry lookahead adder and 4 n-bit registers. Consequently, its area complexity is larger than

$$A = (4*4+1*8)*n=24*n. \tag{6}$$

The product of area and time under this evaluation model is

$$AT = 144*n^2 \log n \tag{7}$$

One possible optimisation of interleaved modular multiplication is given in [12]. The author uses a redundant number representation and carry save adders. His algorithm requires 3 carry save adders and 7 n-bit registers. That means that the area complexity of the algorithm is equal to $3*8n + 7*4n = 52n$ area units. Each iteration of the loop has a time complexity of 3 full adders, thus the time complexity of one iteration is equal to 3. The interleaved modular multiplication requires one loop with n iterations only. Consequently, its time complexity is $3n$ time units. The AT-complexity of this algorithm is equal to $52n * 3n = 106n^2$.

III.4. Optimization of Montgomery

The amount of hardware for the implementation of the loop of Algorithm III.1b can be significantly reduced if we precompute the four possible values to be added to the intermediate result within the loop. There are four cases:

(i) if the sum of the old values of S and C is an even number, and if the actual bit x_i of X is 0, then we don't need to increment S and C at all, i.e. we add 0 before we perform the reduction of S and C by division by 2.

(ii) if the sum of the old values of S and C is an odd number, and if the actual bit x_i of X is 0, then we must add M to make the intermediate result even. Afterwards, we perform the reduction of S and C by division by 2.

(iii) if the sum of the old values of S and C is an even number, and if the actual bit x_i of X is 1, but the increment x_i*Y is even, too, then we don't need to add M to make the intermediate result even. Thus, in the loop we add Y before we perform the reduction of S and C by division by 2. The same action is necessary if the sum of S and C is odd, and if the actual bit x_i of X is 1, and Y is odd as well. In this case, $S+C+Y$ is an even number, too.

(iv) if the sum of the old values of S and C is odd, the actual bit x_i of X is 1, but the increment x_i*Y is even, then we must add Y and M to make the intermediate result even. Thus, in the loop we add $Y+M$ before we perform the reduction of S and C by division by 2. The same action is necessary if the sum of S and C is even, and the actual bit x_i of X is 1, and Y is odd. In this case, $S+C+Y+M$ is an even number, too.

The computation of $Y+M$ can be done prior to going through the loop. This saves one of the two additions which is replaced by the choice of the right operand to be added to the old values of S and C. Algorithm III.3 is a modification of Montgomery's method which takes advantage of this idea:

Algorithm III.3. Faster Montgomery multiplication
Inputs: X, Y, M with $0 \le X, Y < M$
Output: $P = (X \times Y \times (2^n)^{-1}) \bmod M$
n: number of bits in X,
x_i : i^{th} bit of X
s_0: LSB of S, c_0: LSB of C, y_0: LSB of Y
R: precomputed value of $Y+M$
(1) $S := 0$; $C := 0$;
(2) for (i=0: i<k; i++) {
(3) if (($s_0 = c_0$) and not x_0) then $I := 0$;
(4) if (($s_0 \neq c_0$) and not x_0) then $I := M$;
(5) if (not($s_0 \oplus c_0 \oplus y_0$) and x_0) then $I := Y$;
(6) if (($s_0 \oplus c_0 \oplus y_0$) and x_0) then $I := R$;
(7) $S, C := S + C + I$;
(8) $S := S$ div 2; $C := C$ div 2;
(9) $P := S + C$; }
(10) if $P \ge M$ then $P := P - M$;

The operation $\oplus$ in the above algorithm is the "exclusive or operation" (XOR). The advantage of Algorithm III.3 in comparison to Algorithm III.1 can be seen in the implementation of the loop of Algorithm III.3 in Figure III.3. The possible values of I are stored in a lookup-table, which is addressed by the actual values of x_i, y_0, s_0, and c_0. The operations in the loop are now reduced to one table lookup and one carry save addition. Both these activities can be performed concurrently. Note that the shift right operations, which implements the division by 2, can be done by routing.

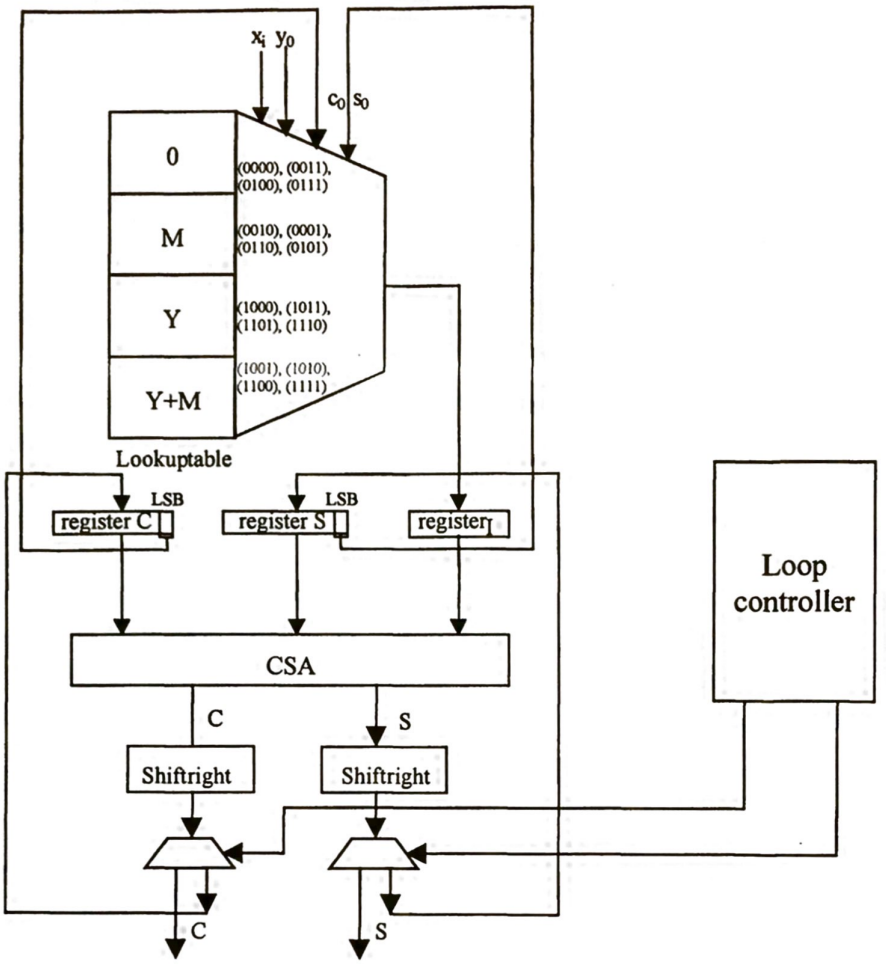


Figure III.3. Implementation of Algorithm III.3

Proposition III.1.

After execution of Algorithm III.3 holds $P = X \times Y \bmod M$.

Proof

It is sufficient to show that the i-th pass of the loop of Algorithm III.3 computes the same values S and C as the corresponding pass i in Algorithm III.1b does. This is obviously the case for $i<0$. We discuss the possible cases for x_i, y_0, s_0, and c_0 for $i \geq 0$:

If $x_i = 0$ and S and C are both even, then Algorithm III.1b leaves S and C unchanged until they are divided by 2. The same holds is S and C are both odd, because the first carry save adder sets s_0 to 0. Exactly the same is done in Algorithm III.3 by assigning 0 to I in step 3.

If $x_i = 0$ and exactly one of S and C is even and the other one is odd, then Algorithm III.1b adds M to make the intermediate result even. We achieve this by assigning M to I in step 4 of Algorithm III.3.

Let now $x_i = 1$. In this case Algorithm III.1b adds Y to S and C in step 3. The result is an even number, exactly if y_0, s_0, and c_0 have been of even parity before this addition. Then Algorithm III.1b does not change the result in step 4, since the new value of s_0 will be 0. The corresponding operation is done in Algorithm III.3. I is set to be Y in step 5 and thus S, C, and Y are added in step 7, producing the same values for S and C as Algorithm III.1b.

If the result after step 3 of Algorithm III.2 is odd, the values of y_0, s_0, and c_0 must have been of odd parity before this addition. In this case the new value of s_0 is 1, and M is added to S and C in step 4. Together S and C are incremented by Y and M in this pass of the loop. The same is done in Algorithm III.3: $R = Y+M$ is assigned to I in step 6, and thus step 7 generates the sum of S, C, Y and M, too. ■

The area requirements of Algorithms III.1b and III.3 can be seen in Figures III.1 and III.3: The predominant area consumers are the adders. Registers for the intermediate results S and C are required in either implementation. We have added one register for I in Algorithm III.3 for simplicity, nevertheless, it could be avoided by performing some minor changes. The registers for Y and M in Algorithm III.1b are replaced by a lookup-table in Algorithm III.3 which is of comparable area, if one takes into account that no units for the computation of x_i*Y and s_0*M are necessary. So, if we neglect the area for the registers the new algorithm reduces the area requirements by a factor of two.

The time performance is not reduced, because it is requires the access of the lookup table instead of one addition. In the loop of Algorithm III.1b the signals have to pass two adders, one after the other, because s_0 must be interpreted to generate one of the operands for the second addition. In Algorithm III.3 only one addition, 3 registers and the lookup table for 4 n-bit values are required and the generation of the subsequent operands can be done concurrently. The area complexity of Algorithm III.3 is equal $8n + 3*4n + 4n = 24n$.Together we get a factor of 1.5 in the product AT of area and time.

III.5. Optimisation of Interleaved Modular Multiplication

The new algorithm is based on the interleaved modular multiplication. But four details of Algorithm III.2 are changed in order to overcome the problems mentioned in Section III.3:

The intermediate results are no longer compared to M (as in steps (6) and (7) of Algorithm III.2). Instead, a comparison to $k*2^n$ (k=0,...,6) is performed which can be done in constant time. This comparison is done implicitly in the mod-operation in step (13) of Algorithm III.4.

Subtractions in steps (6), (7) of Algorithm III.2 are replaced by one subtraction of $k*2^n$ which can be done in constant time by bit masking. Afterwards, the value of $k*2^n$ mod M is added in order to reconstruct the correct intermediate result (step (13) of Algorithm III.4).

The additions are performed by carry save adders reducing the latency to a constant. The intermediate results are in redundant form, coded in two words S and C instead of generated one word P.

An approximate comparison in Algorithm III.2 is sufficient to keep the algorithm correct. Therefore, the comparison is made between the sum of the most significant bits of S and C and the values of k (k=0,...,6) (step (13) in Algorithm III.4).

These modifications lead to Algorithm III.4, which looks more complicated than Algorithm III.2. Its advantage is the fact that all the computations in the loop can be performed in constant time. Thus, the time complexity of the whole algorithm is reduced to $O(n)$, provided the values of $k*2^n \bmod M$ are pre-computed before execution of the loop.

Algorithm III.4. Modular multiplication using carry save addition:
Input: X, Y, M ($X, Y < M$) with $0 \le X, Y \le M$;
Output: $P = X * Y \bmod M$;
(1) $S = 0$;
(2) $C = 0$;
(3) $A = 0$;
(4) for (i=n-1; i≥0; i--) {
(5) $S = S \bmod 2^n$; $C = C \bmod 2^n$;
(6) $S = 2*S$; $C = 2*C$;
(7) $A = 2*A$; $I = x_i*Y$;
(8) $(S, C) = \mathrm{CSA}(S, C, I)$;
(9) $(S, C) = \mathrm{CSA}(S, C, A)$;
(10) $A=(2*s_{n+1}+s_n+2*c_{n+1}+c_n)*2^n \bmod M$;
(11) $P = (S + C) \bmod M$;

Figure III.4 shows the layout of the new algorithm.

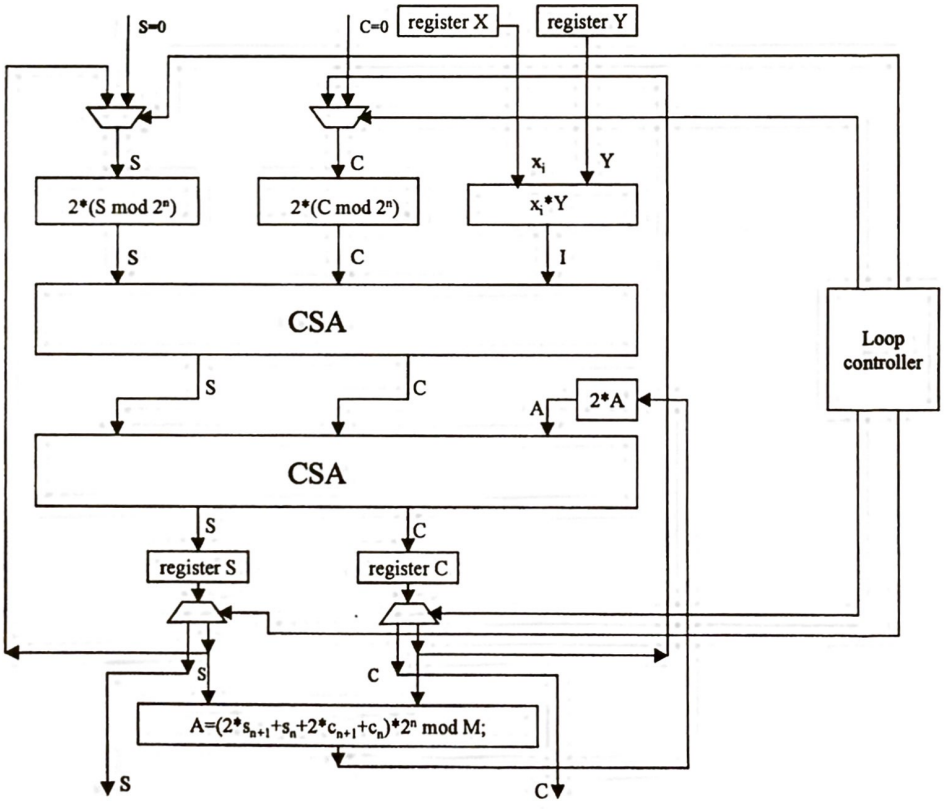


Figure III.4. Inner loop of modular multiplication using carry save addition

We will see that we can significantly speed up and simplify Algorithm III.2 by small modifications. But before, we will prove its correctness:

Proposition III.2

Algorithm III.4 computes the correct value $P = X * Y \bmod M$.

Proof

Let P_k be the value of P after the iteration of the loop in Algorithm III.1 where $i=k$. In the same way we denote the values of S, C, and A in Algorithm III.4 by S_k, C_k, and A_k. We call the iteration where $i=k$ the iteration k. We prove the following invariant:

After iteration k of the loop of Algorithm III.2 and Algorithm III.4 holds

$$P_k = (S_k + C_k + A_k) \bmod M \tag{8}$$

(8) is obviously true for $k=n$, because $P_n = 0$ and S, C, and A are initialised with 0, too. Let (8) hold for $k=i$, $k>1$. Then

$$P_{i-1} \equiv (2P_i + x_{i-1}*Y) \bmod M \equiv (2S_i + 2C_i + 2A_i + x_i*Y) \bmod M \equiv (S'_i + C'_i) \bmod M, \tag{9}$$

where S'_i and C'_i denote the values of S and C after step (12) of the iteration k of Algorithm III.4. In steps (13), (5), and (6) we separate the leading two bits of S and C from the rest.

$$S'_i = (2s_{n+1} + s_n)*2^n + S'_i \bmod 2^n \tag{10}$$

and

$$C'_i = (2c_{n+1} + c_n)*2^n + C'_i \bmod 2^n \tag{11}$$

This implies

$$(C'_i + S'_i) = S'_i \bmod 2^n + C'_i \bmod 2^n + (2s_{n+1} + s_n + 2c_{n+1} + c_n)*2^n \tag{12}$$

And therefore, after steps (5) and (6).

$$P_{i-1} \equiv (S'_i + C'_i) \bmod M \equiv (S_{i-1} + C_{i-1} + A_{i-1}) \bmod M \tag{13}$$

for $i=1$ this means

$$P_0 \equiv (S_0 + C_0 + A_0) \bmod M \equiv X*Y \bmod M \tag{14}$$

Thus, the execution of step (15) of Algorithm III.4 computes the desired result. It remains to show that the values of S and C can never exceed the length of $n+2$ bits. Before the first

carry save addition in step (11) of Algorithm III.4 we know $S<2^{n+1}$, $C<2^{n+2}$, $A<2^{n+1}$, and $I<2^n$. After adding S, C, and I in step (11) it holds $S<2^{n+1}$, $C<2^{n+1}$, and $A<2^{n+1}$

After the addition in step (12) we have $S<2^{n+2}$ and $C<2^{n+2}$, because the addition of the bit in position $n+2$ cannot generate a new carry. ■

We evaluate the complexity of Algorithm III.4 with the same model as in Section III.2. The latency of steps (7) to (10) is not significant because multiplication by 2 is simply a bit-shift and the multiplication in step (10) is performed by a collection of n AND-gates. Steps (11) and (12) require unit time each and step (13) is the access of the lookup table to get access to one of seven precomputed values. Steps (5) and (6) are only bit masking operations which are counted 0 in this model. Summing up, the time for one iteration is 3 and the time for Algorithm III.4 is therefore $3n$. The area is obviously dominated by the two carry save adders of size n each, 4 registers and the lookup table for seven n-bit values adding up to

$$A = (2*8+4*4+7)*n=39n \tag{15}$$

For the area-time complexity we result in

$$T = (39n*3n) = 117n^2 \tag{16}$$

In this calculation we have not measured the effort to compute the values of $k*2^n \bmod M$ (k=0,...,6) in the lookup-table and the final calculation of $(S + C + A) \bmod M$ in step (15). But since the seven values in the lookup-table have to be computed only once for the whole algorithm and the post-computation is also linear in n we don't count it here.

The data flow diagram of Algorithm III.4 can be seen in the hardware layout in Figure III.4.

Algorithm III.5. Optimised version of the new algorithm.
Input: X, Y, M (X, $Y < M$);
Output: $P = X * Y \bmod M$;
LookUp(7) = $(2*3*2^n + Y) \bmod M$; LookUp(6) = $2*3*2^n \bmod M$;
LookUp(5) = $(2*2*2^n + Y) \bmod M$; LookUp(4) = $2*2*2^n \bmod M$;
LookUp(3) = $(2*1*2^n + Y) \bmod M$; LookUp(2) = $2*1*2^n \bmod M$;
LookUp(1) = Y; LookUp(0) = 0;
(1) $S = 0$; $C = 0$;
(2) A = LookUp(x_{n-1});
(3) for (i=n-1; i≥0; i--) {
(4) $S = S \bmod 2^n$;
(5) $C = C \bmod 2^n$;
(6) $S = 2*S$;
(7) $C = 2*C$;
(8) (S, C) = CSA (S, C, A);
(9) A=LookUp($2*(s_n+2*c_{n+1}+c_n)+x_{i-1}$);}
(10) $P = (S + C) \bmod M$;

Figure III.5 shows the architecture for the loop of Algorithm III.5.

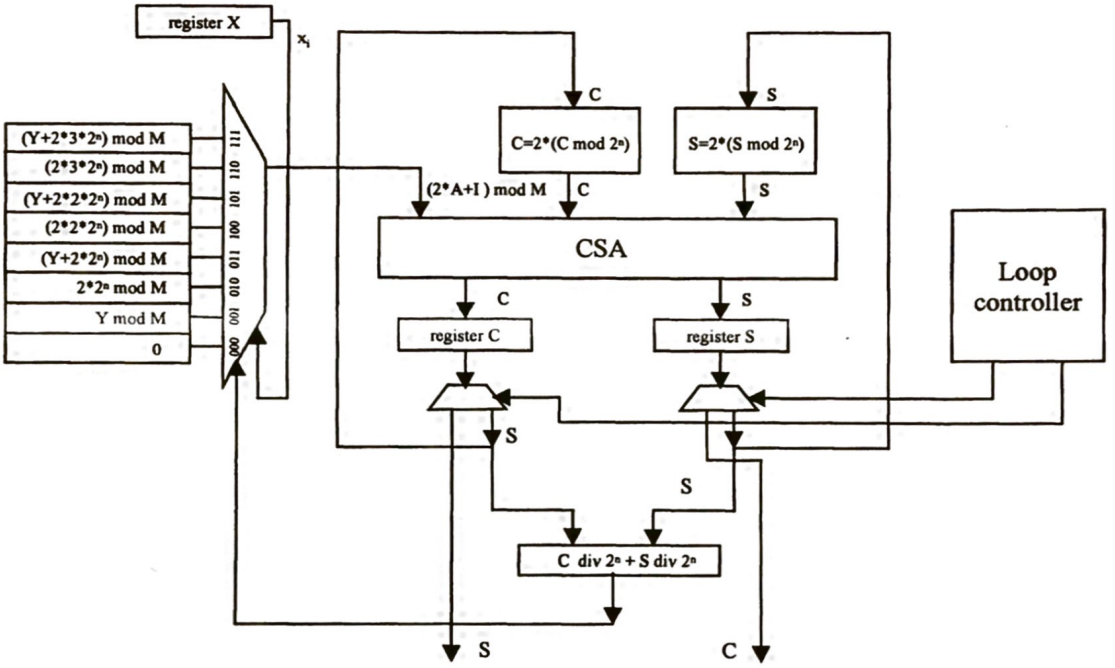


Figure III.5. Modular multiplication with one carry save adder

Now we want to reduce both area and time by further exploiting pre-calculation of values in a lookup-table and thus saving one carry save adder.

The basic idea is simple: the increment I within the loop can have only two possible values: 0 or Y. If x_i is 0 then I is 0 as well. If x_i is 1 then I is Y. Taking this into account we can reduce the number of operands to be added in the loop from four to three. In a pre-computing phase each possible value for $2A+I$ is determined and stored in the lookup-table. During execution of the loop the lookup-table is addressed by the sum of the two leading digits of $S+C$ and by x_i.

The area complexity is now reduced to $(8+3*4+8)*n=28n$ because only one carry save adder is required. The time complexity of one loop iteration is 2 (for the addition and lookup table access). The resulting area-time-complexity of Algorithm III.5 is $56n^2$.

This modification has two positive side effects: Since only three operands are added in the loop, the value for $s_n+2c_{n+1}+c_n$ is always smaller or equal to 3, thus reducing the number of cases. And apart from the lookup-table only three registers are required: one for S, one for C, and one for X.

III.6. SUMMARY

In this section, we want to analyse the different implementations of modular multiplication. We compare four implementations:

Method 1: Montgomery multiplication [10] (Section III.3)
Method 2: Optimised Montgomery multiplication (Section III.4)

Method 3: Interleaved modular multiplication [12] (Section III.3)
Method 4: new algorithm for modular multiplication (Section III.5)

We compare the area complexity: the number of registers and carry save adders, and the time complexity: the number of iteration times the number of the loops times the length of the longest path of the loop.

We can see that Method 2 and Method 4 are significantly better than Method 1 and Method 3, and Method 4 is the most efficient one.

We have shown a new approach to the calculation of modular multiplication. We have compared our new algorithms with the leading algorithms previously known.

This new technique can be further exploited to gain efficiency in many applications. Exponentiation for RSA cryptography can be done with increased speed and reduced hardware complexity. Standard modulo operation and division can be accelerated too. It will be interesting to investigate how much this technique can be further improved by use of higher radix systems to reduce the number of passes through the inner loop of the algorithms.

Table III.1. Performance figures of the described methods

	#registers	#adders	#values of lookup table	area-estimation	longest path	#iterations	time complexity	AT-estimation
Method 1	5	2	0	36n	2	2n	4n	$144n^2$
Method 2	3	1	4	24n	2	2n	4n	$96n^2$
Method 3	7	3	0	52n	3	n	3n	$106n^2$
Method 4	3	1	8	28n	2	n	2n	$56n^2$

III.7. REFERENCES

[1] Bajard, J., Didier, L., Kornerup, P.: "An RNS Montgomery modular multiplication algorithm", *IEEE Transactions on Computers*, Vol. 47, July 1998, pp. 766-776.

[2] Blakley, G. R.: "A Computer Algorithm for Calculating the Product A*B modulo M", *IEEE on Computers*, Vol. C-32, No. 5, May 1983, pp. 497-500.

[3] Blum, T, Paar, C.: "High Radix Montgomery Modular Exponentiation on Reconfigurable Hardware", *IEEE Transactions on Computers*, Vol. 50, No. 7, July 2001, pp. 759-764.

[4] Blum, T., Paar, C.: "Montgomery Modular Exponentiation on Reconfigurable Hardware", *14th IEEE Symposium on Computer Arithmetic (ARITH-14)*, 1999.

[5] Brickell, E.F.: "A Fast Modular Multiplication Algorithm with Application to Two Key Cryptography", *Proc. of Crypto'82*, Plenum Press 1983, pp. 51-60.

[6] Bunimov, V., and Schimmler, M.: "Area and Time Efficient Modular Multiplication of Large Integers", IEEE *International Conference on Application-Specific Systems, Architectures, and Processors (ASAP'03)*, June 2003.

[7] Bunimov, V., Schimmler, M., and Tolg. B.: "A Complexity-Effective Version of Montgomery's Algorithm". *Workshop on Complexity Effective Designs (WCED02)*, May 2002.

[8] Cheng, F-C, Unger, S. H., Theobald, M.: „Self-Timed Carry-Lookahead Adders", *IEEE Transactions on Computers*, Vol. 49, No. 7, July 2000, pp. 659-672.

[9] Eldridge, S.E., Walter, C.D.: "Hardware implementation of Montgomery's modular multiplication algorithm", *IEEE Transaction on Computers*, Vol. 42, July 1993, pp. 693-699.

[10] Kim, Y. S., Kang, W. S., Choi, J. R.: "Implementation of 1024-bit modular processor for RSA cryptosystem", *The 2nd IEEE Asia Pacific Conference on ASICs*, Aug 2000.

[11] Knuth, D. E.: "*The Art of Computer Programming*", Vol. 2, Seminumerical Algorithms. Reading, MA: Addison-Wesley, 2nd printing, p. 423, Nov. 1971.

[12] Koc, C. K.: "RSA Hardware Implementation", RSA *Laboratories, RSA Data Security, Inc*. August 1995, http://security.ece.orst.edu/koc/papers/reports.html.

[13] Lee, S. C.: "*Digital Circuits and Logic Design*", Prentice-Hall, Inc. Englewood Cliffs, New Jersey, 1976.

[14] Menezes et. al: *Handbook of Applied Cryptography*, CRC Press, 1997.

[15] Montgomery, P. L.: "Modular Multiplication Without Trial Division", *Mathematics of Computation*, 44, 1985, pp. 519-521.

[16] Morita, H.: "A Fast Modular-Multiplication Algorithm Based on a Higher Radix", *CRYPTO 1989*, Volume 435, pp. 387-399.

[17] Orton, G. A., Roy, M. P., Scott, P. A., Peppard, L. E. and Tavares, S. E.: "VLSI Implementation of Public-Key Encryption Algorithms", Advances *in Cryptology -- CRYPTO '86*, volume 263, Lecture Notes in Computer Science, Springer-Verlag, 1987, pp. 277-301.

[18] Poldre, J.: "Cryptoprocessor PLD001", *Master Thesis*, Department of Computer science, Tallinn Technical University, June 1998, http://www.pld.ttu.ee/~prj/master.pdf.

[19] Schmidt, B., Schimmler, M., and Adi, W.: "Area Efficient Modular Arithmetic for Mobile Security", *ICWN'02*, Las Vegas, USA (2002).

[20] Shand, M., Vuillemin, J.: "Fast Implementation of RSA Cryptography", *Proc. 11th IEEE Symposium on Computer Arithmetic*, 1993, pp. 252-259.

[21] Sloan, K. R.: "Comments on "A Computer Algorithm for Calculating the Product A*B modulo M"", *IEEE Transactions on Computers*, Vol. C-34, No. 3, March 1985, pp. 290-292.

[22] Walter, C. D.: "Precise Bounds for Montgomery Modular Multiplication and Some Potentially Insecure RSA Moduli", *CT-RSA 2002*: San Jose, CA, USA http://link.springer.de/link/service/series/0558/bibs/2271/22710030.htm.

[23] Wu, C., Chou, Y.: "General Modular Multiplication by Block Multiplication and Table Lookup", *IEEE Int. Symposium on Circuits and System*, ISCAS'94, 1994, pp. 295-298.

[24] Yong-Yin, J., Burleson, W.: "VLSI Array Algorithms and Architectures for RSA Modular Multiplication", *IEEE Transactions on VLSI Systems*, Vol. 5, June 1997, pp. 211-217.

[25] Hong, J.-H., Wu C.-W.: "Radix-4 Modular Multiplication and Exponentiation Algorithms for the RSA Public-Key Cryptosystem", Proceedings of the 2000 conference on Asia South Pacific design automation, 2000, pp. 565 – 570.

Lui[illegible] [illegible]tors)

Embedded Cryptographic Hardware

Methodologies and Architectures

Modern cryptology, which is the basis of information security techniques, started in the late 70's and developed in the 80's. As communication networks were spreading deep into society, the need for secure communication greatly promoted cryptographic research. The need for fast but secure cryptographic systems is growing bigger. Therefore, dedicated systems for cryptography is becoming a key issue for designers. With the spread of reconfigurable hardware such as FPGAs, hardware implementations of cryptographic algorithms become cost-effective.

The focus of this book is on all aspects of embedded cryptographic hardware. Of special interest were contributions that describe secure and fast hardware architectures, new efficient algorithms and promising design methodologies.

ISBN 1-59454-012-8
9 791594 540126